Description of the numerical solver used in the paper*

"Nonperturbative theory of thin-vapor-layer linear optical properties: towards the optimal design of alkali-metal vapor nanocells"

A. V. Ermolaev and T. A. Vartanyan

https://github.com/AndreiErmolaev/TVL-spectra, commit d4bdbd3, October 2026

*This description was prepared with the help of large language models (LLMs) on the basis of the original code of the repository. The authors have carefully checked it against the code and the paper.

How to read this document

This document shows the main steps of the numerical solver used in Ref. [1], as implemented in `tvl.py` of the TVL-spectra repository, along with the corresponding lines of code. The solver computes the reflection, transmission and absorption spectra $R(\Omega)$, $T(\Omega)$ and $A(\Omega)$; `example.ipynb` shows a typical calculation.

- Plain equation numbers, such as (9) or (18), refer to Ref. [1]; numbers with a dot, such as (4.1), are equations of this document.
- **L. 131** means line 131 of `tvl.py`. The listings are read directly from the file and carry its line numbers.

Contents

1 The scheme at a glance 1

2 Self-consistent system of equations for the field and polarization 2

3 Parameters and grid 2

4 The special function J: `I0` and `G` 3

5 The kernel matrix $\boldsymbol{K}$: `build_K` 4

6 The Green matrix $\boldsymbol{S}$ and the wall weights $\mathbf{w}_D$: `build_S_wD` 6

7 Eigenmodes and boundary numbers: `boundary_numbers` 8

8 From boundary numbers to R, T and A 9

9 Accuracy and cost 10

A Derivation of Eqs. (11) and (12) 10

1 The scheme at a glance

The solver computes the two eigenmodes of the vapor layer, even and odd about its centre, exactly in the optical density m, reads off the values and the derivatives of the field at the front wall, and applies the boundary conditions at the two windows. The windows n_1, n_2 enter only in the very last step (Fig. 1).

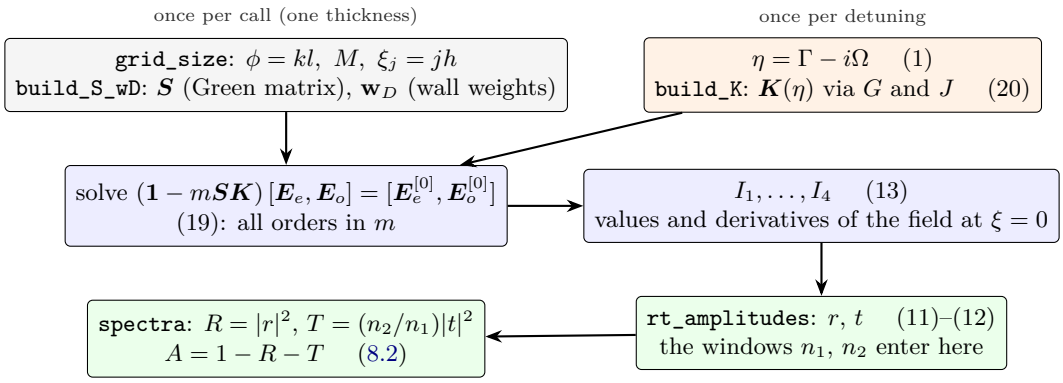


Figure 1: Schematic of the numerical solver. `boundary_numbers` builds the grid and $\boldsymbol{S}$ once, then, for every detuning, $\boldsymbol{K}$, the solution of Eq. (19) and $I_1, \ldots, I_4$; `rt_amplitudes` turns them into r and t, and `spectra` into R, T and A. Plain numbers in parentheses are equations of Ref. [1].

variable	symbol	meaning	Eq.
`l_lam`	l/λ	layer thickness in wavelengths	(1)
`Gamma`	$\Gamma = \gamma/kv_T$	homogeneous half-width in Doppler units	(1)
`m`	$m = 2\sqrt{\pi}Nd^2/\hbar kv_T$	optical density (N: atomic density)	(1)
`Omega, Om`	$\Omega = (\omega - \omega_{21})/kv_T$	detuning in Doppler units; $\Omega > 0$: laser above resonance	(1)
`Gamma-1j*Om`	$\eta = \Gamma - i\Omega$	complex width, the argument `eta` of `build_K` and `G`	(1)
`phi`	$\phi = kl = 2\pi l/\lambda$	phase thickness	(1)
`N`	M	number of grid intervals, even; default `grid_size`	Sec. III B
`xi, h`	$\xi_j = jh$, $h = \phi/M$	grid nodes, $j = 0, \dots, M$	Sec. III B
`mid`	$M/2$	centre node, $\xi_{\mathrm{mid}} = \phi/2$	(9), (15)
`I0`	$J(b)$	$\int_0^\infty e^{-\nu^2 - b/\nu}\,\mathrm{d}\nu$	(20)
`G`	$G(a)$	antiderivative of the kernel $\bar{K}$	(20)
`Gp, Gn`	$G\big((d+\frac{1}{2})h\big)$, $G(dh)$	the values of G used by $\boldsymbol{K}$	—
`K`	$\boldsymbol{K}$	matrix of $g = \hat{\mathcal{K}}E$ (polarization source)	(14), (19)
`S`	$\boldsymbol{S}$	matrix of the Green operator $\hat{\mathcal{S}}$	(15), (19)
`wD`	$\mathbf{w}_D$	weights for the derivatives of the field at the front wall	(6.3)
`seeds`	$E_e^{[0]}, E_o^{[0]}$	$\cos(\xi - \phi/2)$, $\sin(\xi - \phi/2)$	(10)
`Ee, Eo`	E_e, E_o	even and odd eigenmodes	(18), (19)
`I[k]`	(I_1, I_2, I_3, I_4)	boundary numbers, all orders in m	(13)
`n1, n2`	n_1, n_2	front and rear window indices, real	(11), (12)
`r, t`	r, t	reflection and transmission amplitudes	(11), (12)
`R, T, A`	R, T, A	reflected, transmitted and absorbed fractions of the incident flux	(8.2)

Table 1: Variables used in the main code, their mathematical symbols and the corresponding equations of Ref. [1] (plain numbers) or of this document (numbers with a dot).

2 Self-consistent system of equations for the field and polarization

2.1 Model and conventions

In the dimensionless variables of Eq. (1), with $\Omega = (\omega - \omega_{21})/kv_T$ and $\eta = \Gamma - i\Omega$ ($\Omega > 0$ means that the laser is tuned above resonance), the field $E(\xi)$ and the atomic coherences $\sigma(\xi, \nu)$ obey Eqs. (2)–(3),

$$E''(\xi) + E(\xi) = -2im \int_{-\infty}^{\infty} \sigma(\xi, \nu)\, e^{-\nu^2}\,\mathrm{d}\nu, \qquad \nu\,\partial_\xi \sigma + \eta\,\sigma = E, \tag{2.1}$$

with the quenching conditions of Eq. (4), $\sigma(0, \nu > 0) = \sigma(\phi, \nu < 0) = 0$. As shown in Ref. [1], Eqs. (5)–(18), this system reduces to two integral equations for the even and odd eigenmodes of the layer,

$$\begin{aligned} E_e &= E_e^{[0]} + m\,\hat{\mathcal{S}}\hat{\mathcal{K}}\,E_e, \qquad & E_o &= E_o^{[0]} + m\,\hat{\mathcal{S}}\hat{\mathcal{K}}\,E_o, \\ E_e^{[0]}(\xi) &= \cos\Big(\xi - \frac{\phi}{2}\Big), \qquad & E_o^{[0]}(\xi) &= \sin\Big(\xi - \frac{\phi}{2}\Big), \end{aligned} \tag{2.2}$$

which is Eq. (18) with the seeds of Eq. (10). Here $\hat{\mathcal{K}}$ is the convolution with the velocity-averaged kernel $\bar{K}$ of Eq. (7), and $\hat{\mathcal{S}}$ is the Green operator of $d^2/d\xi^2 + 1$ anchored at the centre of the layer, Eqs. (14)–(15). The code solves Eq. (2.2) directly (Sec. 7), which includes all orders in m.

2.2 Boundary numbers, reflection and transmission

The modes enter the continuity conditions at the two walls only through the four boundary numbers of Eq. (13), $I_1 = E_e(0)$, $I_2 = E_o(0)$, $I_3 = E_e'(0)$ and $I_4 = E_o'(0)$, i.e. the values and the derivatives of the field at the front wall; parity supplies the rear wall. Eliminating the amplitudes of the two modes gives the reflection and transmission amplitudes, Eqs. (11)–(12) (Appendix A). The spectra are the reflected, transmitted and absorbed fractions of the incident flux, $R = |r|^2$, $T = (n_2/n_1)|t|^2$ and $A = 1 - R - T$ (Sec. 8).

3 Parameters and grid

`tvl.py` has no global parameters: everything is an argument of `spectra`, documented in the module docstring.

```
Parameters (dimensionless; frequencies in units of the Doppler width k v_T):
    l_lam   layer thickness l / lambda
    Omega   detuning (omega - omega_0) / (k v_T); Omega > 0: laser above resonance
    Gamma   homogeneous half-width gamma / (k v_T)
    m       optical density 2 sqrt(pi) N d^2 / (hbar k v_T)  (Gaussian units)
```

```
    n1, n2  refractive indices of the front window (light comes from it) and of
            the rear window; real, i.e. lossless
    N       number of grid intervals across the layer (even); default grid_size()

Outputs: R = |r|^2, T = (n2/n1) |t|^2, A = 1 - R - T (absorbed
in the vapor).

Typical use (see example.ipynb if you want to directly run the simulations):
    R, T, A = spectra(l_lam=5/3, Omega=Om, Gamma=0.01, m=0.002, n1=1.5, n2=1.5)
    fig, axes = plot_spectra(Om, R, T, A, l_lam=5/3, Gamma=0.01, m=0.002, n1=1.5, n2=1.5)
```

L. 12 l/λ; the phase thickness is $\phi = 2\pi l/\lambda$ (L. 122).

L. 13 $\Omega = (\omega - \omega_{21})/kv_T$, Eq. (1); a scalar or an array. All frequencies are in units of the Doppler width kv_T.

L. 14 $\Gamma = \gamma/kv_T$, Eq. (1).

L. 15 m, Eq. (1). This is the only place the vapor density enters.

L. 16-17 The window indices are not used by the solver. They enter only in `rt_amplitudes` (Sec. 8), and `spectra` rejects complex values (L. 155–156).

L. 18 Number of grid intervals, M in Ref. [1]; even, by default from `grid_size` (below).

L. 20-21 The three outputs, as fractions of the incident flux (Sec. 8).

L. 23-25 The parameters of `example.ipynb`: $l = 5\lambda/3$, $\Gamma = 0.01$, $m = 0.002$, $n_1 = n_2 = 1.5$. The notebook uses 101 detunings in $[-15\Gamma, 15\Gamma]$ ($\Delta\Omega = 0.3\,\Gamma$) and the default grid, $M = 168$.

The grid of Ref. [1], Sec. III B, is set by `grid_size` and built at the top of `boundary_numbers`; the number of grid intervals M is called `N` in the code:

```
def grid_size(l_lam):
    """Default number of grid intervals: even number, about 100 per wavelength, at least 60.
    The error of the spectra is O(h^2), about 1e-5 in R on this grid. Actually, we can go lower..."""
    return max(60, 2 * int(np.ceil(50 * l_lam)))
```

```
    N = grid_size(l_lam) if N is None else int(N)
    if N % 2:
        raise ValueError('N must be even: the centre of the layer must be a grid node')
    phi = 2 * np.pi * l_lam
    xi = np.linspace(0.0, phi, N + 1)
    h = xi[1] - xi[0]
```

L. 66 $M = \max\big(60,\ 2\lceil 50\,l/\lambda\rceil\big)$. The factor 2 makes M **even**, so that the centre $\xi = \phi/2$, where $\hat{\mathcal{S}}$ is anchored, is a grid node. About 100 intervals per wavelength ($h = \phi/M \le 2\pi/100$), and never fewer than 60 intervals for thin layers: $M = 60$ for $l \le 0.6\lambda$, $M = 168$ for the example $l = 5\lambda/3$ and $M = 200$ at $l = 2\lambda$. The error is $O(h^2)$ (Sec. 9).

L. 119 The default grid unless `N` is given; any even M can be forced, for instance to check the convergence by doubling it.

L. 120-121 An odd M is rejected: the centre of the layer must be a node.

L. 122 $\phi = kl = 2\pi\, l/\lambda$, Eq. (1): the layer occupies $0 \le \xi \le \phi$.

L. 123-124 Nodes $\xi_j = jh$, $j = 0, \dots, M$, including both walls ($\xi_0 = 0$, $\xi_M = \phi$), and the spacing $h = \phi/M$.

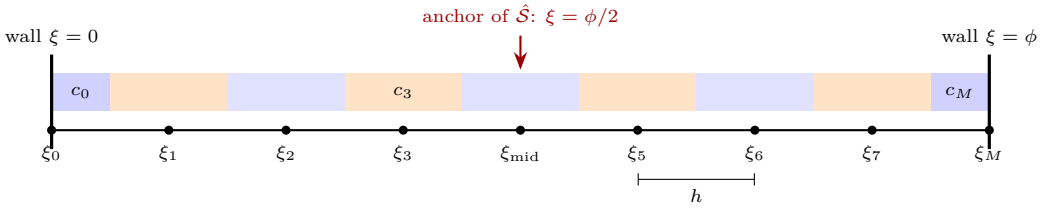


Figure 2: Illustration of the discretization of the vapor layer for $M = 8$ (default grids have $M \ge 60$). Each node ξ_j owns the cell c_j used by $\boldsymbol{K}$: full cells $[\xi_j - h/2, \xi_j + h/2]$ inside, half cells at the walls. The cells cover $[0, \phi]$ exactly.

4 The special function J: `I0` and `G`

4.1 Mathematics

The kernel $\bar{K}$ of Eq. (7) is singular at $s = 0$, so it is never sampled pointwise. Instead it is integrated exactly over each grid cell (Sec. 5) with its antiderivative G of Eq. (20). With $\int_0^a e^{-\eta s/\nu}\,\mathrm{d}s = (\nu/\eta)\big(1 - e^{-\eta a/\nu}\big)$, valid since

$\mathrm{Re}\,\eta = \Gamma > 0$, Eq. (7) gives

$$G(a) \equiv \int_0^a \bar{K}(s)\,\mathrm{d}s = -2i\int_0^\infty \frac{\mathrm{d}\nu}{\nu}\,e^{-\nu^2}\int_0^a e^{-\eta s/\nu}\,\mathrm{d}s = -\frac{2i}{\eta}\int_0^\infty e^{-\nu^2}\big(1-e^{-\eta a/\nu}\big)\,\mathrm{d}\nu$$
$$= -\frac{2i}{\eta}\Big[\frac{\sqrt{\pi}}{2} - J(\eta a)\Big], \tag{4.1}$$

$$J(b) \equiv \int_0^\infty \exp\Big(-\nu^2 - \frac{b}{\nu}\Big)\,\mathrm{d}\nu, \qquad \mathrm{Re}\,b \geq 0, \tag{4.2}$$

which is Eq. (20). Hence $G(0) = 0$, because $J(0) = \sqrt{\pi}/2$, and $G'(a) = \bar{K}(a)$. J is the only special function in the scheme.

Numerical integration. With $\nu = e^t$, $\mathrm{d}\nu = \nu\,\mathrm{d}t$:

$$J(b) = \int_{-\infty}^{\infty} F_b(t)\,\mathrm{d}t, \qquad F_b(t) = \exp\big(-e^{2t} - b\,e^{-t}\big)\,e^t, \tag{4.3}$$

which the code evaluates by the trapezoid rule on $t \in [-13.5,\ 1.9]$ with 4200 points, i.e. for ν from 1.4×10^{-6} to 6.7. The integrand is smooth and negligible beyond both ends, so the rule is very accurate [3]: J is computed to about 10^{-12} for all arguments that occur.

4.2 Code

```
37 def I0(b):
38     """I0(b) = int_0^inf exp(-nu^2 - b/nu) dnu  for complex b with Re b >= 0.
39
40     Trapezoid rule in t = ln(nu) on 4200 points (nu from 1.4e-6 to 6.7) the range
41     is specifically seleceted such that the relative error is about 1e-12."""
42     b = np.atleast_1d(b)
43     t = np.linspace(-13.5, 1.9, 4200)
44     nu = np.exp(t)
45     F = np.exp(-nu**2)[None, :] * np.exp(-np.outer(b, 1.0 / nu)) * nu[None, :]
46     dt = t[1] - t[0]
47     return dt * (F.sum(axis=1) - 0.5 * F[:, 0] - 0.5 * F[:, -1])
48
49
50 def G(a, eta):
51     """G(a) = int_0^a Kern(u) du, the exact antiderivative of the
52     velocity-averaged kernel Kern(u) = -2i int_0^inf exp(-nu^2 - eta u/nu) dnu/nu:
53
54         G(a) = -(2i/eta) [sqrt(pi)/2 - I0(eta a)],   eta = Gamma - i Omega.
55
56     a is a flight distance (in units of 1/k) and G(a) the polarization that
57     atoms starting from zero coherence build up while crossing a distance a of
58     a unit field."""
59     return -(2j / eta) * (SQPI / 2 - I0(eta * np.asarray(a)))
```

- L. 37 The argument `b` may be a scalar or an array of complex numbers b_k; L. 42 makes it at least one-dimensional, and one call evaluates J at all of them.
- L. 43 4200 equally spaced points t_q on $[-13.5, 1.9]$, i.e. ν from 1.4×10^{-6} to 6.7.
- L. 44 $\nu_q = e^{t_q}$.
- L. 45 $F_{kq} = e^{-\nu_q^2}\,e^{-b_k/\nu_q}\,\nu_q = F_{b_k}(t_q)$ of Eq. (4.3). `np.outer(b, 1.0 / nu)` builds b_k/ν_q for all pairs; the last factor ν_q comes from $\mathrm{d}\nu = \nu\,\mathrm{d}t$. `build_K` calls G twice with $M+1$ arguments each, so this line costs $2(M+1)\times4200$ complex exponentials per detuning and dominates the run time (Sec. 9).
- L. 46 Δt.
- L. 47 Trapezoid rule along t: $\Delta t\big(\sum_q F_{kq} - \frac{1}{2}F_{k,0} - \frac{1}{2}F_{k,4199}\big)$, one value per b_k.
- L. 50 $G(a)$ of Eq. (4.1), i.e. Eq. (20). The distance `a` may be an array; `eta` is $\eta = \Gamma - i\Omega$, passed in by `build_K`.
- L. 56–58 The meaning of G: the polarization built up by atoms that start with zero coherence and cross a distance a of a unit field.
- L. 59 Eq. (4.1), with one call of `I0` for all distances. $G(0) = 0$ holds analytically; numerically $J(0)$ carries a tiny integration error, which is why `build_K` never uses $G(0)$ (L. 79).

5 The kernel matrix $\boldsymbol{K}$: `build_K`

5.1 Mathematics: derivation of the matrix elements

The matrix $\boldsymbol{K}$ discretizes $g = \hat{\mathcal{K}}E$, Eq. (14), in three steps.

Step 1: one value of the field per cell. Take $E(s) \approx E_j$ on the cell c_j of node j (Fig. 2): $c_0 = [0, \frac{h}{2}]$, $c_j = [\xi_j - \frac{h}{2}, \xi_j + \frac{h}{2}]$ for $1 \le j \le M-1$, and $c_M = [\phi - \frac{h}{2}, \phi]$. This is the only approximation [2]:

$$g(\xi_i) \approx \sum_{j=0}^{M} \boldsymbol{K}_{ij}\, E_j, \qquad \boldsymbol{K}_{ij} = \int_{c_j} \bar{K}\big(|\xi_i - s|\big)\, \mathrm{d}s, \tag{5.1}$$

so $\boldsymbol{K}_{ij}$ is $g(\xi_i)$ for a unit field confined to the cell c_j.

Step 2: exact cell integral. If c_j lies on one side of ξ_i, the distance $u = |\xi_i - s|$ runs monotonically over an interval $[u_{\text{near}}, u_{\text{far}}]$ with $|\mathrm{d}s| = \mathrm{d}u$, and $G' = \bar{K}$ gives, without approximation,

$$\boldsymbol{K}_{ij} = \int_{u_{\text{near}}}^{u_{\text{far}}} \bar{K}(u)\, \mathrm{d}u = G(u_{\text{far}}) - G(u_{\text{near}}), \tag{5.2}$$

the difference of G at the far and at the near edge of the cell. A cell that contains ξ_i is split at $s = \xi_i$ into two such pieces.

Step 3: the four cases. With $d = |i - j|$, the ranges of u are $[(d - \frac{1}{2})h, (d + \frac{1}{2})h]$ for an interior cell; $[0, \frac{h}{2}]$ twice for the cell around ξ_i, where $G(0) = 0$; $[(i - \frac{1}{2})h, ih]$ for the front half cell, or $[0, \frac{h}{2}]$ when $i = 0$; and the mirror ranges for the rear half cell. Eq. (5.2) then gives

$$\boldsymbol{K}_{ij} = G\big((d + \tfrac{1}{2})h\big) - G\big((d - \tfrac{1}{2})h\big), \qquad d = |i - j| \ge 1, \tag{5.3}$$

$$\boldsymbol{K}_{ii} = 2\, G\big(\tfrac{h}{2}\big), \qquad 1 \le i \le M - 1, \tag{5.4}$$

$$\boldsymbol{K}_{i0} = G(ih) - G\big((i - \tfrac{1}{2})h\big), \qquad 1 \le i \le M, \quad \boldsymbol{K}_{00} = G\big(\tfrac{h}{2}\big), \tag{5.5}$$

$$\boldsymbol{K}_{iM} = G\big((M - i)h\big) - G\big((M - i - \tfrac{1}{2})h\big), \qquad 0 \le i \le M - 1, \quad \boldsymbol{K}_{MM} = G\big(\tfrac{h}{2}\big). \tag{5.6}$$

Figure 3 shows one row of $\boldsymbol{K}$ built this way. The code stores the two families of values needed, $\mathrm{Gp}_d = G\big((d + \frac{1}{2})h\big)$ and $\mathrm{Gn}_d = G(dh)$, $d = 0, \ldots, M$.

Properties.

- Row sums: $\sum_j \boldsymbol{K}_{ij} = G(\xi_i) + G(\phi - \xi_i) = \int_0^\phi \bar{K}(|\xi_i - s|)\, \mathrm{d}s$, so a constant field is integrated exactly.
- Mirror symmetry: $\boldsymbol{K}_{M-i,M-j} = \boldsymbol{K}_{ij}$, i.e. the matrix respects the reflection $\xi \to \phi - \xi$ exactly. $\boldsymbol{K}$ is not symmetric: $\boldsymbol{K}_{i0} \ne \boldsymbol{K}_{0i}$ (half cell against full cell).
- Accuracy $O(h^2)$ for a smooth field.
- $\boldsymbol{K}$ depends on η, i.e. on Ω, and is rebuilt for every detuning.

5.2 Code

```
69  def build_K(eta, N, h):
70      """Matrix of the polarization source, g_i = sum_j K[i, j] E_j, where
71
72          g(xi) = int_0^phi Kern(|xi - s|) E(s) ds.
73
74      E is taken constant on the cell of each node (half cells at the walls) and
75      the kernel is integrated over each cell exactly, so every entry is G(distance
76      to the far cell edge) - G(distance to the near cell edge)."""
77      d = np.arange(N + 1)
78      Gp = G((d + 0.5) * h, eta)                          # G at (d + 1/2) h
79      Gn = G(d * h, eta)                                  # G at d h (Gn[0] is never used)
80      idx = np.abs(np.subtract.outer(d, d))               # distance |i - j| in cells
81      K = Gp[idx] - Gp[np.maximum(idx - 1, 0)]            # interior cells
82      np.fill_diagonal(K, 2 * Gp[0])                      # the cell around node i itself
83      K[:, 0] = np.where(d == 0, Gp[0], Gn[d] - Gp[np.maximum(d - 1, 0)])          # front half cell
84      K[:, N] = np.where(d == N, Gp[0], Gn[N - d] - Gp[np.maximum(N - d - 1, 0)])  # rear half cell
85      return K
```

L. 77 $d = (0, 1, \ldots, M)$. It serves as the distance index for `Gp`, `Gn` and as the row index i in L. 83–84.

L. 78 $\mathrm{Gp}_d = G\big((d + \frac{1}{2})h\big)$: one call of `G`, hence of `I0`, for all d, with the arguments $b = \eta\,(d + \frac{1}{2})h$.

L. 79 $\mathrm{Gn}_d = G(dh)$. The entry $d = 0$, $G(0)$, is never used (see L. 83–84), so the tiny integration error of $J(0)$ does not enter.

L. 80 Matrix of the distances $|i - j|$, in cells.

L. 81 Eq. (5.3) for all entries at once. On the diagonal the index $d - 1$ is clamped to 0, which gives $\mathrm{Gp}_0 - \mathrm{Gp}_0 = 0$, and L. 82 replaces it.

L. 82 Diagonal, Eq. (5.4): the full cell is symmetric about ξ_i, so the singularity is integrated exactly, $2\,G(h/2)$.

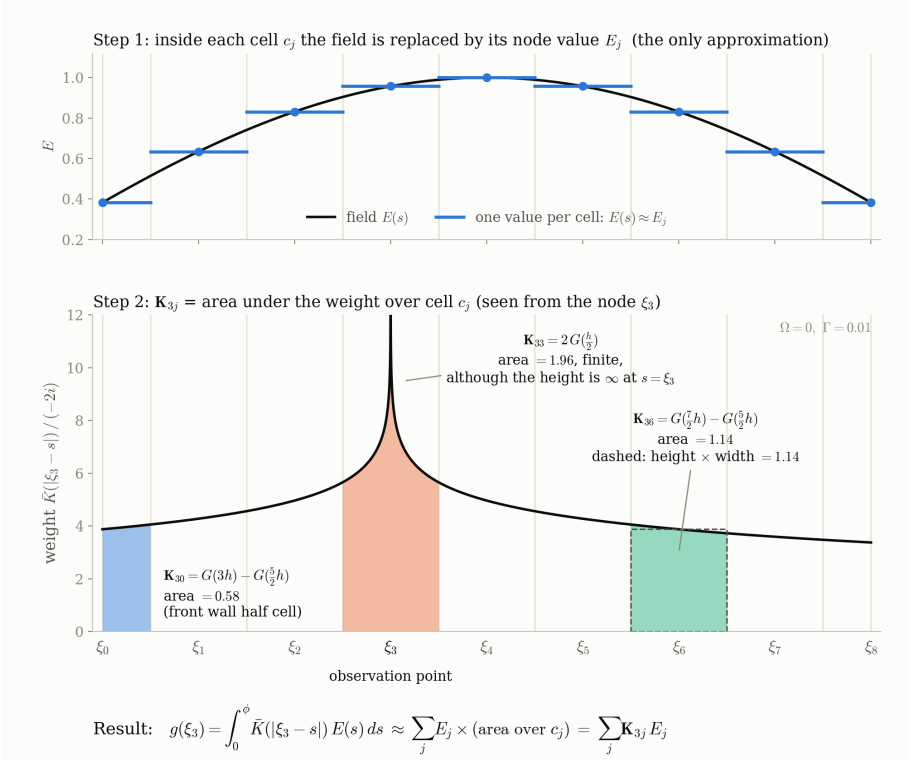


Figure 3: Illustration of the construction of row $i = 3$ of $\boldsymbol{K}$ for $M = 8$, $l = 3\lambda/8$, $\Gamma = 0.01$, $\Omega = 0$ (default grids have $M \geq 60$). Top: the field and its one-value-per-cell approximation, Step 1. Bottom: the weight $\bar{K}(|\xi_3 - s|)/(-2i)$, real and positive at $\Omega = 0$; the shaded areas are $\boldsymbol{K}_{30}$ (front half cell), $\boldsymbol{K}_{33}$ (the cell around the node, where the weight is singular) and $\boldsymbol{K}_{36}$ (an interior cell), Eqs. (5.3)–(5.5).

L. 83 Column $j = 0$, the front half cell, Eq. (5.5). `np.where` selects $\boldsymbol{K}_{00} = G(h/2)$ for $i = 0$; for $i \geq 1$, $\mathrm{Gn}_i - \mathrm{Gp}_{i-1}$.

L. 84 Column $j = M$, the rear half cell, Eq. (5.6), by the mirror formula.

Order matters: L. 82 first sets the corners $\boldsymbol{K}_{00}$, $\boldsymbol{K}_{MM}$ to the full-cell value $2G(h/2)$; L. 83–84 then correct them to the half-cell value $G(h/2)$.

6 The Green matrix S and the wall weights $\mathbf{w}_D$: `build_S_wD`

6.1 Mathematics

Green matrix. Eq. (15) at a node, by the trapezoid rule on the nodes between the anchor and ξ_i (Fig. 4):

$$(\hat{\mathcal{S}}g)(\xi_i) \approx \sum_j \boldsymbol{S}_{ij}\, g_j, \qquad \boldsymbol{S}_{ij} = s_i\, w_j\, \sin(\xi_i - \xi_j), \qquad \begin{cases} j = \mathrm{mid}, \ldots, i, & s_i = +1, \quad i > \mathrm{mid}, \\ j = i, \ldots, \mathrm{mid}, & s_i = -1, \quad i < \mathrm{mid}, \end{cases} \tag{6.1}$$

with $w_j = h$ and $w_j = h/2$ at both ends of the range. For $i < \mathrm{mid}$ the integral runs backwards, $\int_{\phi/2}^{\xi_i} = -\int_{\xi_i}^{\phi/2}$, hence $s_i = -1$. The row $i = \mathrm{mid}$ is zero, since $\hat{\mathcal{S}}g$ vanishes at the anchor, and so is the diagonal, since $\sin 0 = 0$. Entries outside the range between the anchor and ξ_i are zero as well, so only about a quarter of $\boldsymbol{S}$ is non-zero (Fig. 4, Step 3). The error is $O(h^2)$. Like $\boldsymbol{K}$, $\boldsymbol{S}$ has the mirror symmetry $\boldsymbol{S}_{M-i,M-j} = \boldsymbol{S}_{ij}$ but is not symmetric.

Derivative of the field at the wall. Differentiating Eq. (15) gives $(\hat{\mathcal{S}}g)'(\xi) = \int_{\phi/2}^{\xi} \cos(\xi - \xi')\, g(\xi')\, \mathrm{d}\xi'$; the boundary term vanishes because $\sin 0 = 0$. For $E = E^{[0]} + m\,\hat{\mathcal{S}}g$ with $g = \hat{\mathcal{K}}E$, Eq. (2.2), the derivative of the field at the front wall is therefore

$$E'(0) = E^{[0]\prime}(0) + m \int_{\phi/2}^{0} \cos(-s)\, g(s)\, \mathrm{d}s = E^{[0]\prime}(0) - m \int_{0}^{\phi/2} \cos s\; g(s)\, \mathrm{d}s, \tag{6.2}$$

discretized by the trapezoid rule on nodes $0, \ldots, \mathrm{mid}$:

$$E'(0) \approx E^{[0]\prime}(0) + m\, \mathbf{w}_D \cdot g, \qquad (\mathbf{w}_D)_j = \begin{cases} -w_j \cos \xi_j, & 0 \leq j \leq \mathrm{mid}, \\ 0, & j > \mathrm{mid}, \end{cases} \tag{6.3}$$

with $E_e^{[0]\prime}(0) = \sin(\phi/2)$ and $E_o^{[0]\prime}(0) = \cos(\phi/2)$. This differentiates the integral representation exactly instead of differencing the computed field, which would be less accurate.

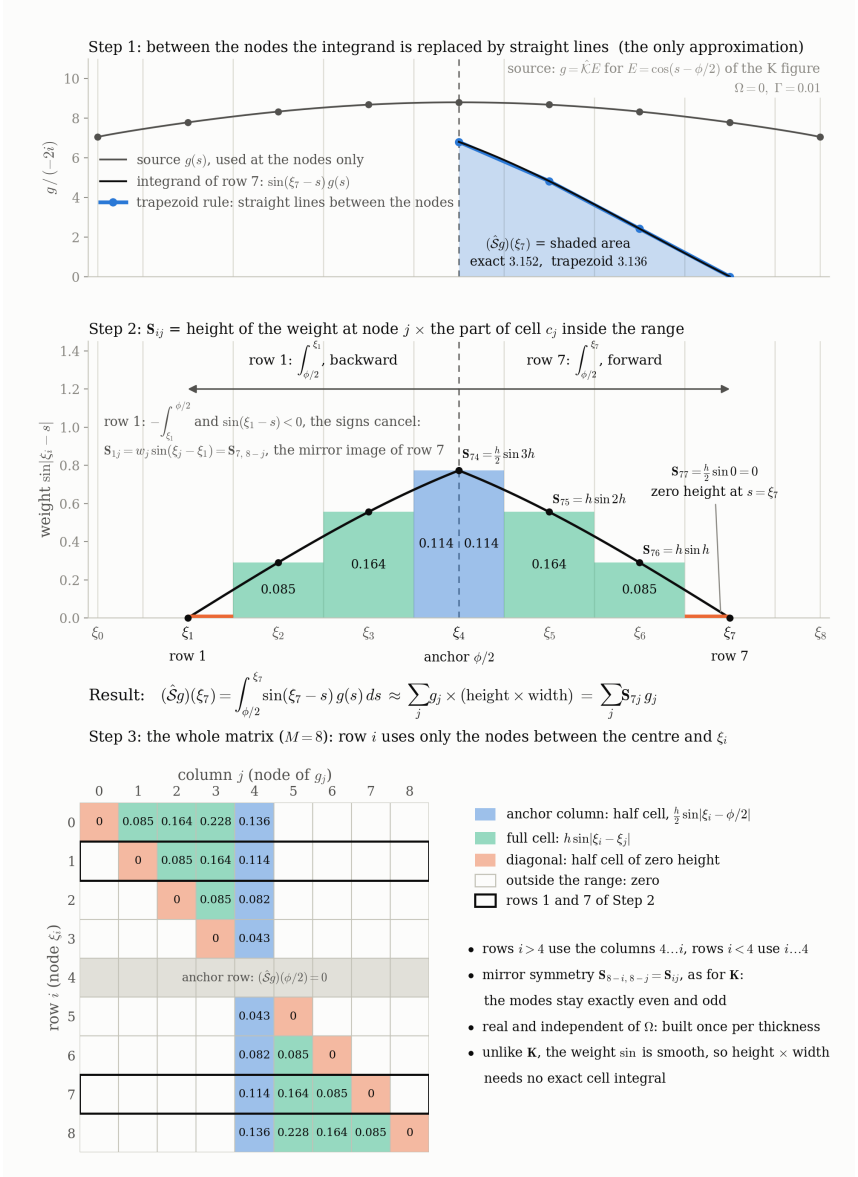


Figure 4: Illustration of the construction of $\boldsymbol{S}$ for $M = 8$, $l = 3\lambda/8$ (default grids have $M \geq 60$). Step 1: the integrand of row 7 between the anchor and ξ_7 and the straight lines of the trapezoid rule, for the source $g = \boldsymbol{K}E$ of the field of Fig. 3 at $\Omega = 0$, $\Gamma = 0.01$. Step 2: the weights of rows 7 and 1 as height × width over the cells c_j of $\boldsymbol{K}$ cut to the integration range. Step 3: the whole matrix.

6.2 Code

```
def build_S_wD(N, h, xi):
    """Matrix S and front-wall weights wD (trapezoid rule):

        (S g)(xi_i) = int_{phi/2}^{xi_i} sin(xi_i - s) g(s) ds,
        E'(0) = E0'(0) + m wD . g  with  wD . g = -int_0^{phi/2} cos(s) g(s) ds.

    S is anchored at the centre of the layer, which fixes the parity of the modes."""
    mid = N // 2
    S = np.zeros((N + 1, N + 1), dtype=complex)
    for i in range(N + 1):
        if i == mid:
            continue                                  # (S g) vanishes at the anchor
        j0, j1, sgn = (mid, i, 1.0) if i > mid else (i, mid, -1.0)
        w = np.full(j1 - j0 + 1, h)
        w[0] = w[-1] = h / 2
        S[i, j0:j1 + 1] = sgn * w * np.sin(xi[i] - xi[j0:j1 + 1])
    w = np.full(mid + 1, h)
    w[0] = w[-1] = h / 2
    wD = np.zeros(N + 1)
    wD[:mid + 1] = -w * np.cos(xi[:mid + 1])
    return S, wD
```

L. 95 Index of the centre node, $\xi_{\mathrm{mid}} = \phi/2$; M is even (L. 120–121).

L. 96 $(M+1)\times(M+1)$ zeros. $\boldsymbol{S}$ is real but stored as complex, so that the product $\boldsymbol{SK}$ needs no type conversion.

L. 97-99 Loop over rows; the centre row stays zero (anchor).

L. 100 Integration range and sign of Eq. (6.1): forward from the centre for $i >$ mid, backward ($s_i = -1$) for $i <$ mid.

L. 101-102 Trapezoid weights h, halved at both ends of the range.

L. 103 Row i of $\boldsymbol{S}$. The entry $j = i$ is multiplied by $\sin 0 = 0$; the columns outside the range keep their zeros.

L. 104-107 $\mathbf{w}_D$ of Eq. (6.3): trapezoid weights on $[0, \phi/2]$ times $-\cos\xi_j$. The minus sign and the cosine of Eq. (6.2) are already included.
Cost $O(M^2)$, once per call of `boundary_numbers` (L. 125).

7 Eigenmodes and boundary numbers: `boundary_numbers`

7.1 Mathematics

All orders in m. With $\hat{\mathcal{K}} \to \boldsymbol{K}$ and $\hat{\mathcal{S}} \to \boldsymbol{S}$, the two equations (2.2) become the linear system (19) of Ref. [1], with the seeds of Eq. (10) as the two right-hand sides:

$$(\mathbf{1} - m\,\boldsymbol{SK})\begin{bmatrix}\boldsymbol{E}_e & \boldsymbol{E}_o\end{bmatrix} = \begin{bmatrix}\boldsymbol{E}_e^{[0]} & \boldsymbol{E}_o^{[0]}\end{bmatrix}. \tag{7.1}$$

Expanding $(\mathbf{1} - m\,\boldsymbol{SK})^{-1}$ in powers of m reproduces the series (17) term by term, so one direct solve of Eq. (7.1) includes all orders in m. The solve does not need the series to converge, only the matrix $\mathbf{1} - m\boldsymbol{SK}$ to be invertible. The boundary numbers of Eq. (13) follow from the solution vectors and Eq. (6.3):

$$\begin{aligned} I_1 &= (\boldsymbol{E}_e)_0, & I_2 &= (\boldsymbol{E}_o)_0, \\ I_3 &= \sin\frac{\phi}{2} + m\,\mathbf{w}_D\cdot\boldsymbol{KE}_e, & I_4 &= \cos\frac{\phi}{2} + m\,\mathbf{w}_D\cdot\boldsymbol{KE}_o, \end{aligned} \tag{7.2}$$

where $(\cdot)_0$ is the value at node 0, the front wall.

7.2 Code

```
112 def boundary_numbers(l_lam, Omega, Gamma, m, N=None):
113     """Values and slopes at the front wall of the even and odd eigenmodes,
114     (I1, I2, I3, I4) = (Ee(0), Eo(0), Ee'(0), Eo'(0)) [their Eq. (34)], for every
115     detuning. The modes solve (1 - m S K) E = E0 with the seeds
116     E0 = cos(xi - phi/2) and sin(xi - phi/2), i.e. exactly in m.
117
118     Returns a complex array of shape (len(Omega), 4)."""
119     N = grid_size(l_lam) if N is None else int(N)
120     if N % 2:
121         raise ValueError('N must be even: the centre of the layer must be a grid node')
122     phi = 2 * np.pi * l_lam
123     xi = np.linspace(0.0, phi, N + 1)
124     h = xi[1] - xi[0]
125     S, wD = build_S_wD(N, h, xi)                    # independent of the detuning
126     seeds = np.column_stack([np.cos(xi - phi / 2), np.sin(xi - phi / 2)]).astype(complex)
127     Omega = np.atleast_1d(np.asarray(Omega, dtype=float))
128     I = np.empty((Omega.size, 4), dtype=complex)
129     for k, Om in enumerate(Omega):
130         K = build_K(Gamma - 1j * Om, N, h)          # eta = Gamma - i Omega
131         Ee, Eo = np.linalg.solve(np.eye(N + 1) - m * (S @ K), seeds).T
132         I[k] = (Ee[0], Eo[0],
133                 np.sin(phi / 2) + m * (wD @ (K @ Ee)),
134                 np.cos(phi / 2) + m * (wD @ (K @ Eo)))
135     return I
```

L. 112 The arguments are the thickness, the detuning(s), Γ, m and optionally the number of intervals `N`. The windows are not arguments: the boundary numbers do not depend on n_1, n_2.

L. 119-124 Grid (Sec. 3).

L. 125 $\boldsymbol{S}$ and $\mathbf{w}_D$ depend on the grid only, not on Ω, so they are built once per call; only $\boldsymbol{K}(\eta)$ is rebuilt for every detuning.

L. 126 The seeds $E_e^{[0]} = \cos(\xi - \phi/2)$ and $E_o^{[0]} = \sin(\xi - \phi/2)$ of Eq. (10), i.e. Eq. (9) with $(C_1, C_2) = (1, 0)$ and $(0, 1)$, as the two columns of one complex array: the two right-hand sides of Eq. (7.1).

L. 127 The detunings as a one-dimensional float array, so a scalar works too.

L. 128 Output: one row $(I_1, \ldots, I_4)$ per detuning.

L. 129-130 Loop over detunings; $\eta = \Gamma - i\Omega$, Eq. (1), and $\boldsymbol{K}(\eta)$ (Sec. 5), the only matrix that depends on Ω.

L. 131 Eq. (7.1), i.e. Eq. (19): `np.linalg.solve` solves for both modes at once, and `.T` splits the result into the even and the odd mode on the nodes. Cost $O(M^3)$.

L. 132-134 $I_1, \dots, I_4$, Eq. (7.2): the values at node 0 (the front wall) and the derivatives of the field from Eq. (6.3), with $g = \boldsymbol{K}\boldsymbol{E}_e$ and $\boldsymbol{K}\boldsymbol{E}_o$.

L. 135 Return the boundary numbers; the windows have not appeared yet.

8 From boundary numbers to R, T and A

8.1 Mathematics

With $P = I_1 I_4 + I_2 I_3$, $Q = I_1 I_2$ and $U = I_3 I_4$, Eqs. (11)–(12) read

$$r = \frac{(n_1 - n_2)\,P + 2i\,(n_1 n_2\,Q + U)}{(n_1 + n_2)\,P + 2i\,(n_1 n_2\,Q - U)}, \qquad t = \frac{2n_1\,(I_1 I_4 - I_2 I_3)}{(n_1 + n_2)\,P + 2i\,(n_1 n_2\,Q - U)}. \tag{8.1}$$

Appendix A derives them from the continuity conditions at the two walls and the parity relations of Eq. (13). The spectra are fractions of the incident flux:

$$R = |r|^2, \qquad T = \frac{n_2}{n_1}\,|t|^2, \qquad A = 1 - R - T. \tag{8.2}$$

Empty cell. At $m = 0$ the modes are the seeds, so $(I_1, I_2, I_3, I_4) = (\cos\frac{\phi}{2}, -\sin\frac{\phi}{2}, \sin\frac{\phi}{2}, \cos\frac{\phi}{2})$. Then $P = \cos\phi$, $Q = -\frac{1}{2}\sin\phi$, $U = \frac{1}{2}\sin\phi$, $I_1 I_4 - I_2 I_3 = 1$, and

$$r_0 = \frac{(n_1 - n_2)\cos\phi - i(n_1 n_2 - 1)\sin\phi}{(n_1 + n_2)\cos\phi - i(n_1 n_2 + 1)\sin\phi}, \qquad t_0 = \frac{2n_1}{(n_1 + n_2)\cos\phi - i(n_1 n_2 + 1)\sin\phi}, \tag{8.3}$$

the standard Fabry–Pérot result for an empty gap between the two windows [4]; $R_0 + T_0 = 1$ for real n_1, n_2, which the code reproduces to machine precision.

8.2 Code

```
138 def rt_amplitudes(I, n1, n2):
139     """Reflection and transmission amplitudes r, t from the boundary numbers.
140     Note that the the windows only enter in this part, as the computation before was
141     done to compute the eigenmodes of the vapor layer."""
142     I1, I2, I3, I4 = I[..., 0], I[..., 1], I[..., 2], I[..., 3]
143     P, Q, U = I1 * I4 + I2 * I3, I1 * I2, I3 * I4
144     den = (n1 + n2) * P + 2j * (n1 * n2 * Q - U)
145     r = ((n1 - n2) * P + 2j * (n1 * n2 * Q + U)) / den
146     t = 2 * n1 * (I1 * I4 - I2 * I3) / den
147     return r, t
148
149
150 def spectra(l_lam, Omega, Gamma, m, n1, n2, N=None):
151     """Reflectance R, transmittance T and absorptance A versus detuning.
152
153     Parameters: see the module docstring. Omega may be a scalar or an array.
154     Returns three real arrays (fractions of the incident flux, not %)."""
155     if np.iscomplexobj(n1) or np.iscomplexobj(n2):
156         raise ValueError('n1 and n2 must be real: T and A are defined for lossless windows')
157     r, t = rt_amplitudes(boundary_numbers(l_lam, Omega, Gamma, m, N), n1, n2)
158     R = np.abs(r) ** 2
159     T = (n2 / n1) * np.abs(t) ** 2
160     return R, T, 1.0 - R - T
161
162
163 def empty_cell(l_lam, n1, n2):
164     """Reflectance R0 and transmittance T0 of the empty cell (m = 0, Fabry-Perot).
165     Without atoms the modes are the seeds, so the boundary numbers are those of
166     cos(xi - phi/2) and sin(xi - phi/2) at xi = 0. l_lam may be an array."""
167     phi = 2 * np.pi * np.asarray(l_lam, dtype=float)
168     I = np.stack([np.cos(phi / 2), -np.sin(phi / 2), np.sin(phi / 2), np.cos(phi / 2)], axis=-1)
169     r, t = rt_amplitudes(I, n1, n2)
170     return np.abs(r) ** 2, (n2 / n1) * np.abs(t) ** 2
```

L. 138-141 The only place the windows enter. The expensive solve is therefore independent of n_1, n_2: spectra for other windows need only this function, about a microsecond per detuning.

L. 142 Unpack $I_1, \dots, I_4$ from the last axis; `I` may hold many rows, such as (detunings, 4) here or (thicknesses, 4) in `empty_cell`.

L. 143 P, Q, U.
L. 144 The common denominator of Eq. (8.1).
L. 145 r, Eq. (11).
L. 146 t, Eq. (12); $I_1I_4 - I_2I_3$ equals 1 for the empty cell.
L. 155–156 Complex indices are rejected: T and A as flux fractions require lossless windows.
L. 157 The whole chain: `boundary_numbers` (Sec. 7), then `rt_amplitudes`.
L. 158–160 Eq. (8.2): R, T and A as fractions, not per cent.
L. 163–170 Empty cell, Eq. (8.3): the boundary numbers of the seeds at $\xi = 0$ in closed form (L. 168), then the same `rt_amplitudes`; `l_lam` may be an array. It returns R_0 and T_0; $A_0 = 0$.

8.3 Plotting

`plot_spectra` (L. 182–199) draws R, T and A in per cent against $\Omega/\Gamma = (\omega - \omega_{21})/\gamma$, with the parameters in the title, and returns the figure; `example.ipynb` uses it to save `figures/example_spectra.png`. Matplotlib is imported inside this function only, so the solver itself needs only NumPy.

9 Accuracy and cost

- **Grid.** The spatial discretization is the only approximation that matters. Its error falls as h^2, i.e. by a factor of about 4 each time M is doubled. On the default grid the spectra are accurate to a few 10^{-5} (absolute, in R, T and A); doubling `N` is the simplest check.
- **Velocity integral.** The numerical integration of J (function `I0`) is accurate to about 10^{-12}.
- **Optical density.** There is no truncation in m: the solve includes all orders.
- **Cost.** On a laptop one detuning takes about 0.04 s for $M = 60$ and 0.1–0.13 s for $M = 168$–200, most of it (over 80 %) in the computation of J inside `build_K`; the 101 detunings of `example.ipynb` take about 10 s.

A Derivation of Eqs. (11) and (12)

Write the field in the layer as $E = aE_e + bE_o$. With the parity relations of Eq. (13), the continuity of the field and of its derivative at the two walls reads

$$\begin{aligned} \text{front wall:}\quad & 1 + r = aI_1 + bI_2, & in_1(1-r) &= aI_3 + bI_4, \\ \text{rear wall:}\quad & t = aI_1 - bI_2, & in_2\,t &= -aI_3 + bI_4. \end{aligned}$$

Eliminating t from the two rear-wall conditions gives $b = a\,(I_3 + in_2I_1)/(I_4 + in_2I_2)$. Inserting this into the front-wall conditions, with $D = I_4 + in_2I_2$:

$$1 + r = \frac{a}{D}\,(P + 2in_2Q), \qquad in_1(1-r) = \frac{a}{D}\,(2U + in_2P).$$

The ratio eliminates a/D:

$$\frac{in_1(1-r)}{1+r} = \frac{2U + in_2P}{P + 2in_2Q} \quad\Longrightarrow\quad r = \frac{i(n_1 - n_2)P - 2(n_1n_2Q + U)}{i(n_1 + n_2)P - 2(n_1n_2Q - U)},$$

and multiplying numerator and denominator by $-i$ gives Eq. (11). Adding the two front-wall conditions instead, after dividing the second by in_1, eliminates r:

$$2 = \frac{a}{D}\Big[P + 2in_2Q + \frac{2U + in_2P}{in_1}\Big] \quad\Longrightarrow\quad \frac{a}{D} = \frac{2n_1}{(n_1 + n_2)P + 2i(n_1n_2Q - U)}.$$

The rear wall gives $t = aI_1 - bI_2 = (a/D)\big[I_1(I_4 + in_2I_2) - I_2(I_3 + in_2I_1)\big] = (a/D)(I_1I_4 - I_2I_3)$, which is Eq. (12). `spectra` turns t into the transmitted flux $T = (n_2/n_1)|t|^2$ (Sec. 8).

References

[1] A. V. Ermolaev and T. A. Vartanyan, "Nonperturbative theory of thin-vapor-layer linear optical properties: towards the optimal design of alkali-metal vapor nanocells" (2026).

[2] K. E. Atkinson, The Numerical Solution of Integral Equations of the Second Kind (Cambridge University Press, Cambridge, 1997).

[3] L. N. Trefethen and J. A. C. Weideman, "The exponentially convergent trapezoidal rule", SIAM Rev. **56**, 385 (2014).

[4] M. Born and E. Wolf, Principles of Optics, 7th ed. (Cambridge University Press, Cambridge, 1999), Sec. 1.6.